

Introduction to Machine Learning

Jacques Bahi

Table Of Content I

1 Machine Learning

2 Machine Learning and Deep Learning

Definition of Machine Learning I

We rely on Machine Learning every day : Social networks - Transportation - Healthcare - Weather forecasting - Booking platforms - etc.

Machine Learning models are computer algorithms that use data (huge datasets) to make predictions.

These uses are impossible to achieve with traditional programming because it would require coding billions of possible cases ; this is where Machine Learning comes in.

We give the machine the ability to **learn** from data. We **train** the machine on data.

Definition of Machine Learning II

Machine Learning models differ from traditional algorithms in how they are designed :

- A traditional computer software uses a function that we program to generate outputs based on inputs.
- A machine learning algorithm is capable of learning from data.

Definition of Machine Learning III

We don't explicitly program a function F .

The machine's parameters are set by the data itself. The larger the dataset, the better the prediction or classification.

A simple example is spam filtering using machine learning. Twenty years ago, spam filters didn't have many examples they could learn from, so they weren't very good at identifying what was or wasn't spam. As the amount of spam data became enormous and

Definition of Machine Learning IV

was reported as such by human users, machine learning algorithms gained experience to perform spam/not spam classification.

Two classic types of problems in machine learning :

- Classification problems : Given k categories (classes), the computer program is prompted to specify which of the k categories certain inputs belong to.
- Regression problems : In this type of task, the computer program is prompted to predict a numerical value using certain input.

The issue at hand is different from classical mathematical or computer science problems : it is the data that determines the parameters of the model, not the model function that determines the correspondence between inputs and outputs.

A properly trained model will be able to predict and classify with good performance : it provides good prediction or classification when presented with **new inputs**.

Vocabulary :

We have a **dataset**. The inputs/outputs we have are called "**training variables**". We train the model until we estimate that it predicts/classifies correctly.

To test the model, we create a subset of inputs/outputs that the model never "sees" during training. These are the "**test variables**". When the model performs well with unseen data, we say it "**generalizes**".

How to measure the validity and performance of an ML model?

By introducing what is called a "**loss function**" that measures the gap between predicted value and desired value (target). The gap is a distance between the predicted output vector and the desired output. Euclidean distance is an example of a distance used (Mean Square Error).

The more "ground" data we have, the more the model learns and adjusts, the more performant it becomes. Hence the emergence of the Big Data domain. ML applications continuously collect data via the web. ChatGPT, 45 terabytes of compressed data.

The massive and exponential quantities of data and the need for near real-time responses, the algorithmic complexity of the models, the non-trivial calculations explain why scientific libraries such as **Numpy**, **Matplotlib**, or **Pandas** have been developed and included in

specialized Machine Learning (AI) frameworks :

- scikitlearn
- Pytorch,
- tensorflow,
- Keras,
- NLTK, spaCy

All under the Python programming language.

NumPy : Arrays and matrix computations I

NumPy, short for Numerical Python, is one of the most important fundamental packages for numerical computing in Python. Most computational packages that provide scientific functionality use NumPy's array objects as a common language for data exchange.

This is the case for **Pandas**, **Matplotlib**, **Seaborn** and so on ...

The first tutorials will aim to review the main features of these three packages.

Below are some of the features provided by NumPy :

NumPy : Arrays and matrix computations II

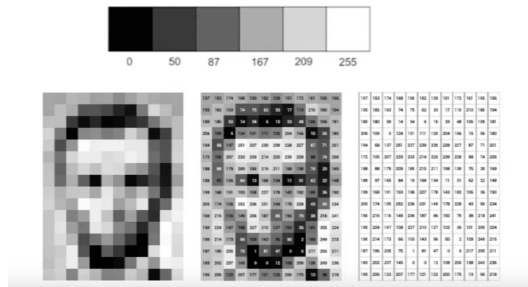


Figure – Image NB

NumPy : Arrays and matrix computations III

3D array

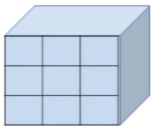


Figure – Image Couleur

NumPy : Arrays and matrix computations IV

- **ndarray**, an efficient multidimensional array providing fast array-oriented arithmetic operations and flexible broadcasting capabilities.
- Mathematical functions for fast operations on entire data arrays.
- Tools for reading/writing array data to disk and working with memory-mapped files.
- Capabilities for linear algebra, random number generation, and more.
- A C API for connecting NumPy to libraries written in C, C++, or Fortran.

NumPy : Arrays and matrix computations V

Since NumPy provides an easy-to-use C API, it is straightforward to pass data to external libraries written in a low-level language and for those libraries to return data to Python as NumPy arrays.

This feature has made Python a language of choice for wrapping legacy C/C++/Fortran codebases and giving them a dynamic, user-friendly interface.

One of the main reasons NumPy is so important for numerical computing in Python is that it is designed to be efficient on large data arrays.

NumPy : Arrays and matrix computations VI

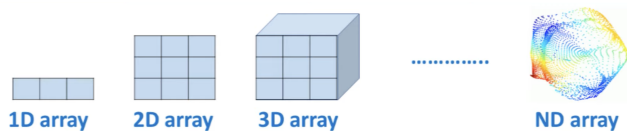


Figure – Ndarray

NumPy : Arrays and matrix computations VII

1. Constructeur:
 - `np.array`
 - `np.zeros`
 - `np.random.randn`
 - ...
2. **Shape**, **ndim**, **size**
3. Dtype: **float16**, **float64**
4. Manipulation:
 - **Concatenate**, **hstack**, **vstack**
 - **Reshape**, **squeeze**
 - **Ravel**

Operations on arrays :

```
1 In [12] : import numpy as np
2 In [13] : data = np.random.randn(2, 3)
3 In [14] : data
4 Out[14] :
5 array([[ -0.2047,  0.4789, -0.5194],
6        [-0.5557,  1.9658,  1.3934]])
```

Some basic mathematical operations on arrays :

```
1 In [15] : data * 10
2 Out[15] : array([[ -2.0471,  4.7894, -5.1944],
3                [-5.5573,  19.6578,  13.9341]])
4 In [16] : data + data
```

```
5 Out[16] : array([[ -0.4094,  0.9579, -1.0389],  
6          [-1.1115,  3.9316,  2.7868]])
```

```
7
```

```
8
```

Matrix computaion

```
1 In [51]: arr = np.array([[1., 2., 3.], [4., 5., 6.]])
2 In [52]: arr
3 Out[52]:
4 array([[ 1.,  2.,  3.],
5        [ 4.,  5.,  6.]])
6 In [53]: arr * arr
7 Out[53]:
8 array([[ 1.,  4.,  9.],
9        [16., 25., 36.]])
10 In [54]: arr - arr
11 Out[54]:
12 array([[ 0.,  0.,  0.],
```

```
13 [ 0.,  0.,  0.]])
```

```
1 In [55]: 1/arr
```

```
2 Out[55]:
```

```
3 array([[ 1. , 0.5 , 0.3333],
```

```
4 [ 0.25 , 0.2 , 0.1667]])
```

```
5 In [56]: arr ** 0.5
```

```
6 Out[56]:
```

```
7 array([[ 1.      ,  1.4142,  1.7321],
```

```
8 [ 2.      ,  2.2361,  2.4495]])
```



```
1 import numpy as np
2
3 a = np.array([[1, 4],
4 [5, 6]])
5
6 b = np.array([[2, 4],
7 [5, 2]])
8
9 print(np.dot(a, b))
10
11 Out: [[22 12]
12       [40 32]]
```

Reshape- Concatenate I

Reshape- Concatenate II

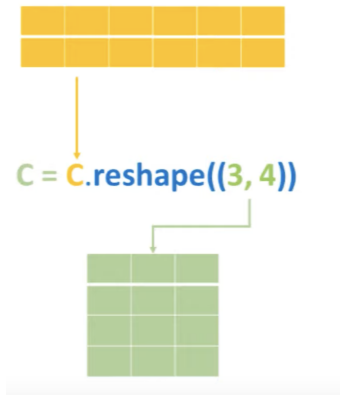


Figure – Reshape

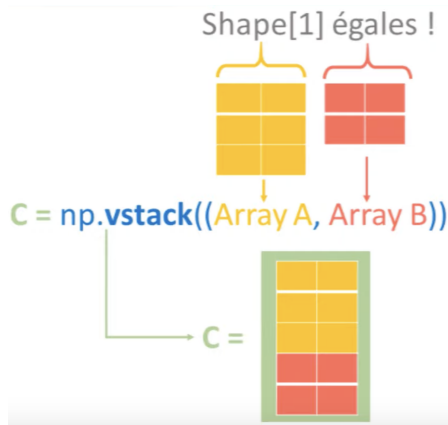
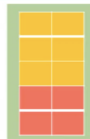


Figure – Vstack

```
C = np.vstack((Array A, Array B))  
np.concatenate((Array A, Array B), axis = 0)
```



```
C = np.hstack((Array A, Array B))  
np.concatenate((Array A, Array B), axis = 1)
```

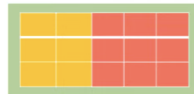
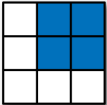
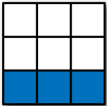
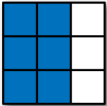


Figure – H et V stack vs Concatenate

Slicing I

Slicing II

	Expression	Shape
	<code>arr[:2, 1:]</code>	<code>(2, 2)</code>
	<code>arr[2]</code>	<code>(3,)</code>
	<code>arr[2, :]</code>	<code>(3,)</code>
	<code>arr[2:, :]</code>	<code>(1, 3)</code>
	<code>arr[:, :2]</code>	<code>(3, 2)</code>
	<code>arr[1, :2]</code>	<code>(2,)</code>
	<code>arr[1:2, :2]</code>	<code>(1, 2)</code>

Pandas : Data Importing and Cleaning I

The Pandas library heavily relies on the NumPy library, as its data structures are based on the NumPy arrays (ndarrays). Thus, in addition to the ndarray structure, the Pandas library also benefits from the computational performance of NumPy.

The advantage of Pandas over NumPy is that the DataFrame object can store data of different types, i.e. heterogeneous data. This means that in a dataframe, we can have a column containing numbers, another containing text, another containing booleans, etc.

Pandas : Data Importing and Cleaning II

The ability to store heterogeneous data comes from the fact that a **dataframe** is made up of a set of independent **series** and can therefore contain different data types from one to another. However, within a column, the data must be of the same type. Furthermore, dataframes and series come with the ability to assign labels to the data, rather than working with numerical indices, as was the case with NumPy ndarrays. With Pandas, rows and columns can be identified with labels rather than numbers.

DataFrame : ~ Dictionnaire de Séries

Rappel: Dict ['clef'] = valeur

Df ['column'] = une Série

	pclass	survived	sex	age
0	1	1	female	29.0000
1	1	1	male	0.9167
2	1	0	female	2.0000
3	1	0	male	30.0000
4	1	0	female	25.0000

Figure – Series

```
1 data = {'state': ['Ohio', 'Ohio', 'Ohio', 'Nevada', 'Nevada', 'Nevada'],  
         'year': [2000, 2001, 2002, 2001, 2002, 2003], 'pop': [1.5, 1.7,
```

```
3.6, 2.4, 2.9, 3.2]}}
```

```
2 frame = pd.DataFrame(data)
```

```
3 In [45]: frame
```

```
4 Out[45]:
```

```
5 pop    state  year
```

```
6 0  1.5    Ohio  2000
```

```
7 1  1.7    Ohio  2001
```

```
8 2  3.6    Ohio  2002
```

```
9 3  2.4  Nevada  2001
```

```
10 4  2.9  Nevada  2002
```

```
11 5  3.2  Nevada  2003
```

```
12 6.
```

The head method returns the first five rows :

```
1 In [46]: frame.head()
```

```
2 Out[46]:
```

```
3
```

```
4 pop      state  year
```

```
5 0   1.5      Ohio  2000
```

```
6 1   1.7      Ohio  2001
```

```
7 2   3.6      Ohio  2002
```

```
8 3   2.4  Nevada  2001
```

```
9 4   2.9  Nevada  2002
```

```
1
```

```
2
```

```
3 # You can change the positions of the columns :
4 In [47]: pd.DataFrame(data, columns=['year', 'state', 'pop'])
5 Out[47]:
6
7 year    state  pop
8 0   2000    Ohio  1.5
9 1   2001    Ohio  1.7
10 2   2002    Ohio  3.6
11 3   2001  Nevada  2.4
12 4   2002  Nevada  2.9
13 5   2003  Nevada  3.2
```

```
1 # We can extract a column:
```

```
2
3 In [51]: frame['state']
4 Out[51]:
5 one      Ohio
6 two      Ohio
7 three    Ohio
8 four     Nevada
9 five     Nevada
10 six     Nevada
11 # an equivalent way
12 In [52]: frame.year
```

```
1 # We can also extract rows, by the use of \texttt{loc} function:
```

```
2 In [53]: frame.loc[3]
3 Out[53]:
4 year      2001
5 state      Nevada
6 pop        2.4
7 Name: three, dtype: object
```

Importing and reading txt and csv files I

The first step is usually to import the data into your notebook.

Files can be in different formats, but the most common in Data Science is the CSV (comma-separated values) format. It is a text format, in which columns are separated by commas.

Another very common format is the TXT format, which is also a text format, but containing values separated by tabs. To read a TXT file with Pandas, you need to use the `read_table()` function.

The general syntax is as follows :

```
1  import pandas as pd
2  pd.read_table("myfile.txt")
```

As seen before, to read a CSV file with Pandas, you use the `read_csv()` function. Note that the `read_table()` and `read_csv` functions can adapt to any separator, whether it's semicolon (;), space (" "), this is done through the `sep` argument as follows :

```
1  import pandas as pd
2  pd.read_table("myfile.csv", sep=",")
```

Let's import an example dataset into our notebook, we read the data file with the `readcsv()` function, and store it in a dataframe named *data* :

```
1
2  import pandas as pd
3  data=pd.read_csv("./datasets/data_file.csv")
4  data.head()
```

The Pandas `head()` method, which applies to a series or a dataframe, allows you to have a preview of the first five rows of our object to see what it looks like. You will often use this method, as in data analysis, we work with large data tables and it is neither advisable nor useful to display them in full.

When looking at a data table, we generally see that there is an ID column that corresponds to identification. We could define it as an index.

For this, we can use the following code :

```
1 import pandas as pd
2 indexed_data=pd.read_csv("../datasets/myfile.csv", index_col=[0])
3 indexed_data.head()
```

Filtering columns when reading the file :

```
1 import pandas as pd
2 pd.read_csv("myfile.csv", usecols=[3,6])
```

Here we are asking to import only columns 3 to 6.

Reading Excel files :

```
1 import pandas as pd
2 excel_file=pd.read_excel("myfile.xlsx")
```

Reading JSON files

```
1 import pandas as pd
2 pd.read_json("../datasets/file.json")
3
```

Each key-value pair represents a column of the row. The Pandas library provides a function called `read_json()` which allows reading a JSON file. JSON format is widely used for exchanging data in a structured and lightweight manner. JSON format is a text format, but its fields are organized differently. All data is stored within braces `[]`. Then, each individual or observation is stored within parenthesis `()`. The parenthesis can be compared to rows of a traditional table. Finally, each parenthesis contains all the information (columns) of the individual/observation in the form of key-value pairs separated by commas.

Indexing and Slicing with iloc Attribute I

The iloc attribute allows selecting values based on their positions in rows and columns.

If we want to select a value at a specific row and column, the syntax is as follows :

```
1 dataframe.iloc[26,7]
```

To select values from multiple rows and columns, we need to provide a list of row names and a list of column names :

```
1 dataframe.iloc[[26, 65], [7,12]]
```

To select a single row and all columns, the syntax is :

```
1 dataframe.iloc[26,:]
```

To select multiple rows and all columns, we need to provide a list of row positions as follows :

```
1 dataframe.iloc[[26, 65], :]
```

Finally, to select a specific column and all rows, the syntax is :

```
1 dataframe.iloc[:,7]
```

Slicing can be performed to select slices of rows and columns.

```
1 dataframe.iloc[0:1, 0:10]
```

Here, we select the first row and the first ten columns of the dataframe.

Before exploring the data in a table, it needs to be cleaned and prepared.

There are two important steps to consider beforehand : handling missing values, and if necessary, handling duplicated values.

Handling Missing Data I

When data cannot be listed, when the value in the dataframe is empty, its value becomes NaN. It is important to be aware of the proportion of missing values, and even to handle them, as we will see in this section.

The general syntax for counting the number of missing values in a dataframe is :


```
1 my_dataframe.isna().sum()
```

The `isna()` method will replace all values in the dataframe with `True` (if the value is `NaN`) or `False` (if the value is different from `NaN`). Then the `sum()` method will count the number of `True` values per column of the dataframe, which will allow us to quantify the presence of missing values. Deleting Missing Values

To delete rows containing a `NaN` in at least one column or all columns, there is the `dropna()` method.

```
1 dataframe.dropna(how="any")
```

The `how="any"` option specifies to the `dropna()` method that if a single value in the row is equal to `NaN`, then that row should be deleted.

It is also possible to delete rows if all the values are NaN, that is, if the NaN value is present in all columns, using the `how="all"` option.

```
1 dataframe.dropna(how="all")
```

Finally, to delete rows with a missing value in a specific column, you will need to use the `notna()` method rather than the `dropna()` method.

```
1 dataframe[dataframe['my_column'].notna()]
```

Replacing Missing Values I

It may happen that we don't want to delete rows containing missing values, for fear of losing information, but we still want to have no missing values in the table. It is then possible to use a method called `fillna()`, which will replace the NaN values as requested. For example, if we want to replace all NaN with 0, the syntax is as follows : `dataframe.fillna(value=0, inplace=True)` We can also replace NaN values with the mean of the column in which the value is located, with the following syntax :

```
1 dataframe[['column1', 'column4']]=(dataframe[['column1', 'column4',  
    ]].fillna(dataframe[['column1', 'column4']].mean()))
```

Detecting/Recovering Duplicate Rows I

Duplicated information can be duplicated entire rows. The general syntax for detecting entirely duplicated rows in a table is `duplicated()`.

The `duplicated()` method will return a series containing booleans, allowing to say whether the row is duplicated (True) or not (False).

By default, the `duplicated()` method keeps the first occurrence of duplicated rows and deletes the others, thanks to the `keep = 'first'` option. This returns a dataframe containing

Detecting/Recovering Duplicate Rows II

all duplicated rows except the first occurrence, which `duplicated()` leaves as `False` to avoid losing information.

```
1  # detection:
2  dataframe[dataframe.duplicated()]
3
4  # Deleting duplicate rows
5  dataframe.drop_duplicates(inplace=True)
```

Performing Descriptive Statistics I

The `describe()` method generates descriptive statistics to explore, in the case of quantitative variables, the distribution of data within the variables. It also allows generating descriptive statistics on qualitative variables.

By default, the `describe()` method returns statistics for quantitative variables.

The `describe()` method with the `include="all"` option returns statistics for both quantitative and qualitative variables.


```
1  # Default
2  dataframe.describe()
3
4  # Entire statistics
5  data.describe(include="all")
```

Matplotlib I

Matplotlib is a plotting package designed to create high-quality graphs. The matplotlib community has collaborated to simplify interactive plotting from the IPython interpreter and now, the Jupyter notebook (we will use Jupyter notebook).

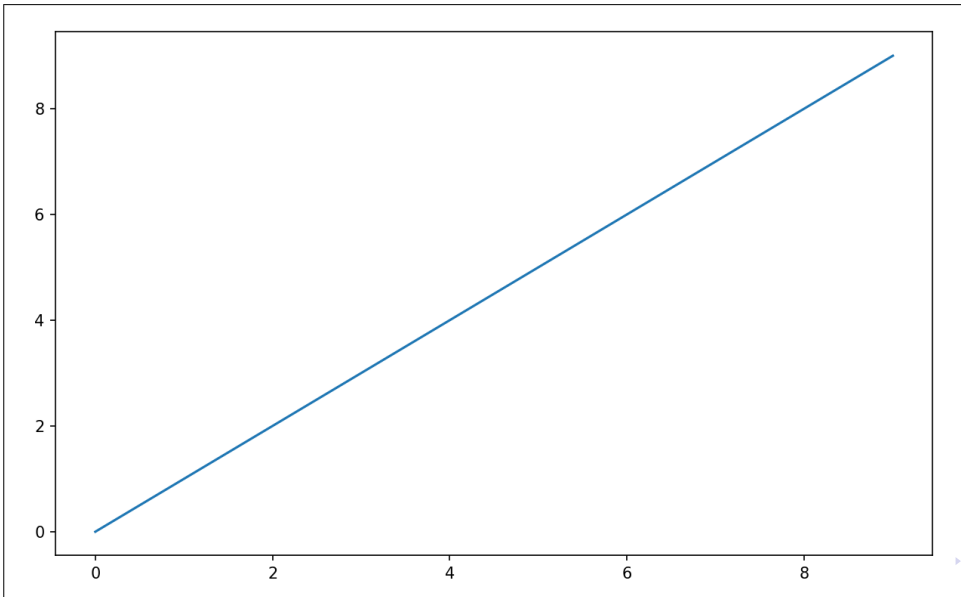
Matplotlib supports various graphical backends across all operating systems and can also export visualizations to all common vector and raster graphics formats (PDF, SVG, JPG, PNG, BMP, GIF, etc.).

Matplotlib II

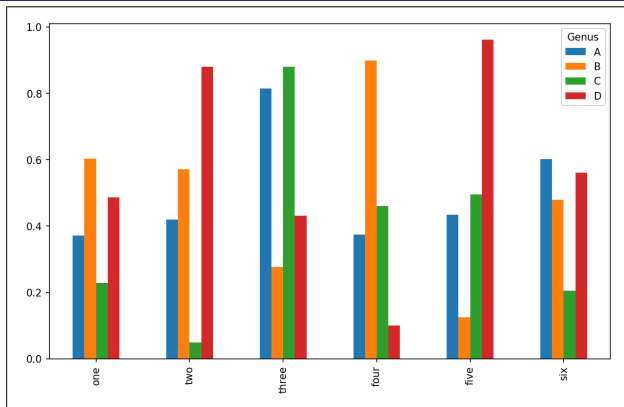
Over time, matplotlib has spawned several toolkits complementary to data visualization that use matplotlib for their underlying plotting. One of them is **seaborn**.

Simple examples

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  data = np.arange(10)
4  # print the data
5  data
6  # Plot
7  plt.plot(data)
8  plt.show()
```



```
1 df = pd.DataFrame(np.random.rand(6, 4),  
2 index=['one', 'two', 'three', 'four', 'five', 'six'],  
3 columns=pd.Index(['A', 'B', 'C', 'D'], name='Genus'))  
4 df.plot.bar()
```

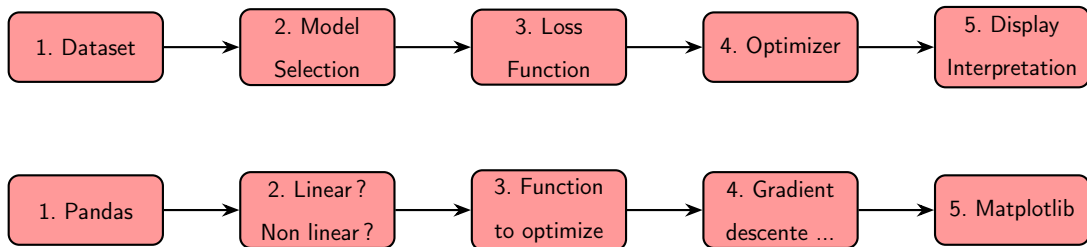


Macro steps in Machine Learning Software I

- 1 **Dataset Preparation and Preprocessing** : We work with Python using Pandas – Seaborn which allow us to read and clean any type of data : txt, csv, Jason, Excel, SQL database, etc.
- 2 **Model Selection** : Linear model ? Non-linear ? Statistical ? Neural network model ? The model depends on **parameters**.
- 3 **Loss Function** : To calculate the errors made during predictions and adjust them through training.

Macro steps in Machine Learning Software II

- 4 **Optimizer Selection** : Algorithm that minimizes the loss function. It searches for the best **parameters** for minimal cost : Gradient Descent ? Random Forest ? SVM ? ...
- 5 **Display and Interpretation of Results** : Pandas – Matplotlib and good sense of analysis.



Choosing the Optimizer I

AI specialized environments integrate different optimizers, for example :

- Scikit-learn is a free Python library for machine learning. It is developed by numerous contributors, notably by French higher education and research institutes such as Inria. It offers many algorithm libraries to implement, ready-made. These libraries are available to data scientists in particular. It is designed to harmonize with other free Python libraries, notably NumPy.

Choosing the Optimizer II

- TensorFlow is an open-source machine learning tool developed by Google. It has an interface for Python, Julia, and R. TensorFlow is one of the most widely used tools in AI in the field of machine learning.
- The Keras library allows interaction with deep neural network algorithms and machine learning, notably TensorFlow and Microsoft Cognitive Toolkit.
- PyTorch is an open-source Python software library for machine learning that builds on Torch developed by Meta.

Machine Learning and Deep Learning I

There are mainly two types of learning :

- **Supervised Machine Learning** models are trained with labeled datasets, allowing models to learn and become more accurate over time. For example, an algorithm would be trained with images of dogs and other things, all labeled by humans, and the machine would learn to identify images of dogs on its own. Supervised machine learning is the most commonly used type today.

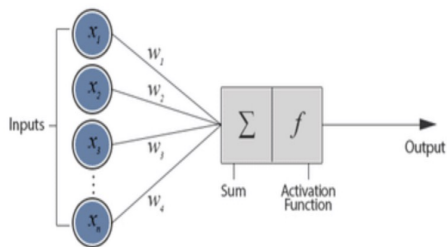
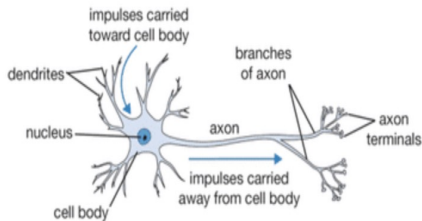
Machine Learning and Deep Learning II

- **Unsupervised Machine Learning** : a program looks for patterns in unlabeled data. Unsupervised machine learning can find patterns or trends that people are not explicitly looking for. For example, one could go through online sales data and identify different types of customers making purchases.

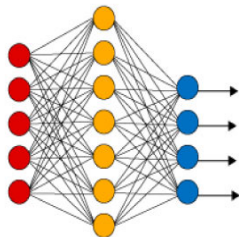
Deep learning : While Machine Learning works with smaller datasets and requires manual feature extraction, Deep learning automatically learns complex patterns from raw data like images, sound and text and performs better with large datasets.

Deep Learning

Biological Neuron versus Artificial Neural Network

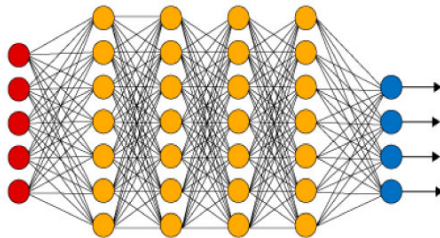


Simple Neural Network



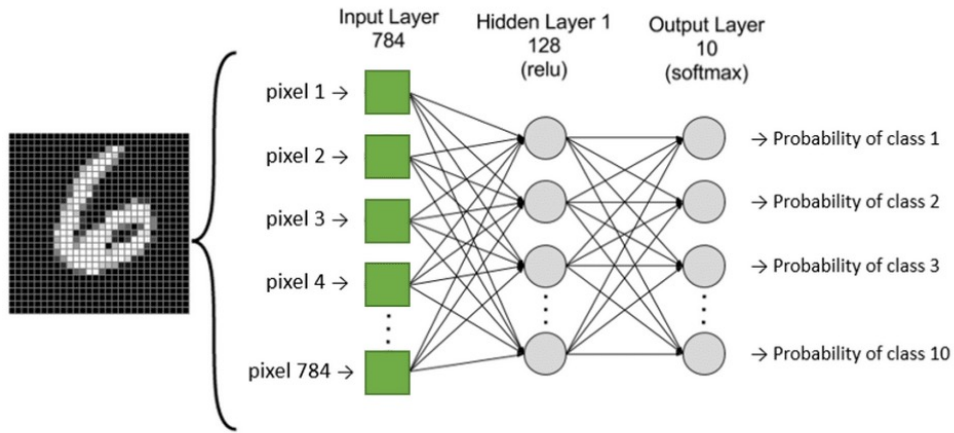
● Input Layer

Deep Learning Neural Network

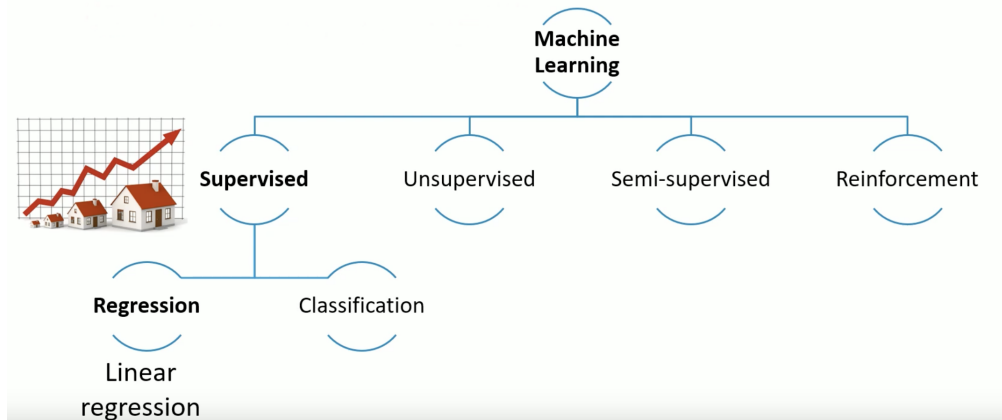


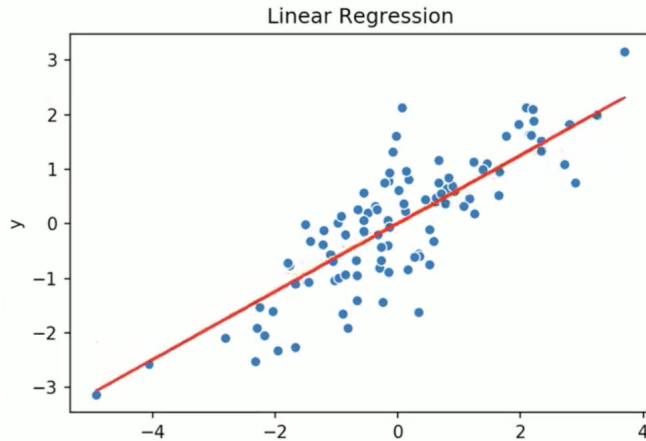
● Hidden Layer

● Output Layer



Machine Learning. Regression I





Machine Learning. Regression I

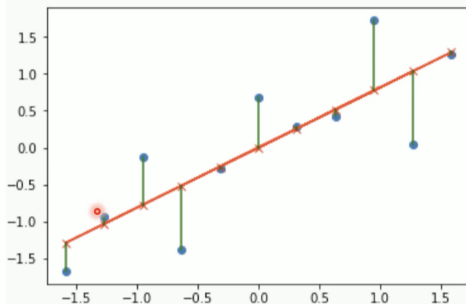
Train

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Cost function

Residuals sum of errors =

$$\frac{1}{2m} \sum (h_{\theta}(x(i)) - y(i))^2$$



We can then move on to the step : Choice of optimizer.

Machine Learning. Classification I

Classification is a common task in Machine Learning. It involves putting a label on an observation :

- Is a tumor malignant or not ?
- Is a transaction fraudulent or not ?

Machine Learning. Classification II

When there are 2 possible label choices (malignant or benign tumor), it is called Binary Classification : the label Y will have two possible values 0 or 1. In other words, $Y \in \{0, 1\}$. Example of classification method in Machine Learning : logistic regression. Some regression algorithms can be used for classification. Logistic regression (also called logit regression) is commonly used to estimate the probability that an observation belongs to a particular class.

Another example : what is the probability that this email is spam ?

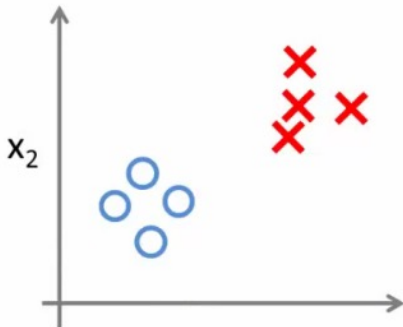
Machine Learning. Classification III

- If the estimated probability is greater than 50%, then the model predicts that the observation belongs to this class (called the positive class, labeled "1").
- Otherwise, it predicts that it belongs to the other class (the negative class, labeled "0"). It's actually a binary classifier.

Machine Learning. Classification IV

Logistic regression is a supervised classification algorithm, popular in Machine Learning.

Binary classification:



Machine Learning. Classification I

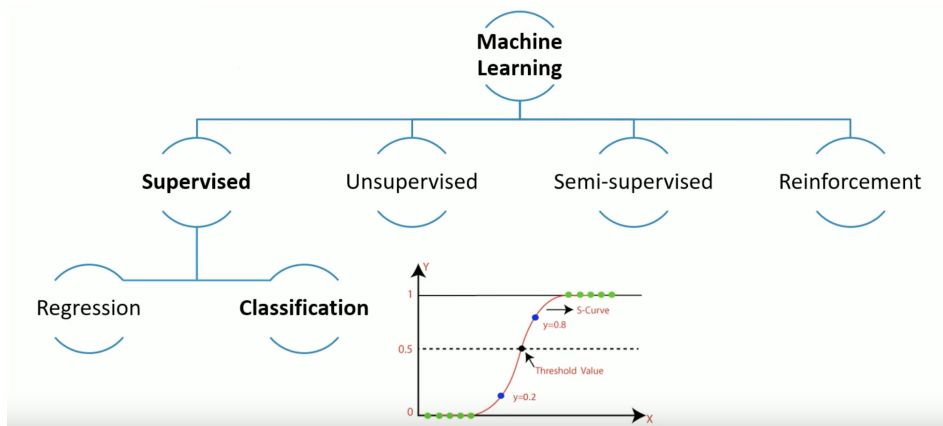
We will detail its operation for binary classification and then we will see its generalization to multi-class classification. The goal is to find a line separating the two groups (Decision Boundary). For the example of the tumor, we can arbitrarily assign class (label) 1 to say that it is a malignant tumor and value 0 for benign tumors.

When our problem has multiple possible labels (for example, classifying an article into a category ("sport", "politics", "High-Tech")...), we talk about : Multi-class classification.

Machine Learning. Classification II

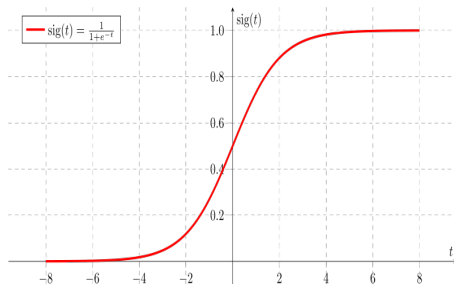
In this case, $Y \in \{1, 2, 3, \dots\}$. Again, we can arbitrarily assign class numbers to observations in the Training Set (Training Data).

Machine Learning. Classification III



Machine Learning. Classification I

The Logistic function is defined as $\sigma(x) = \frac{1}{1+e^{-x}}$

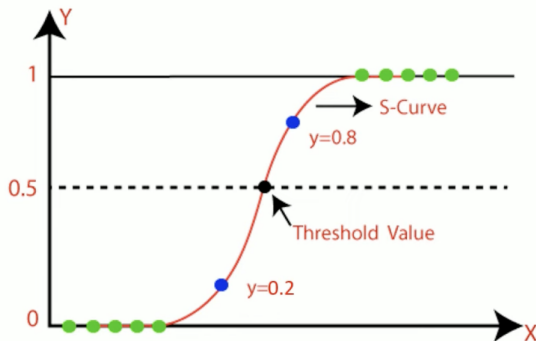


Machine Learning. Classification II

We can notice that :

- It passes through the ordinate 0.5.
- It has 2 asymptotes at $y = 0$ and $y = 1$.
- $\sigma(x) > 0.5$ then x is classified in 1) and $\sigma(x) < 0.5$ then x is classified in 0).

If we take the example of tumors, if $H(X)$ returns 0.7, this means that the patient has a 70% chance of having a cancerous tumor. In other words, he has a 30% chance of having a benign tumor. More formally, the probability that X belongs to class (label) 1 with parameters θ_i is : $P(y = 1|X, \theta) = H(X)$.



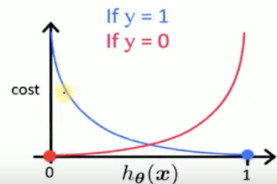
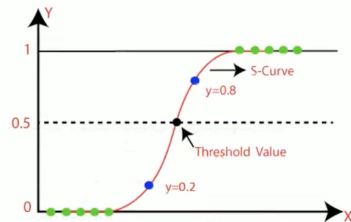
Train (déterminer la fonction sigmoïde)

$$h_{\theta}(x) = g(z) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$$

Cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$



$$J^{(i)} = -\log p_{\theta}(y = y^{(i)} | x^{(i)})$$

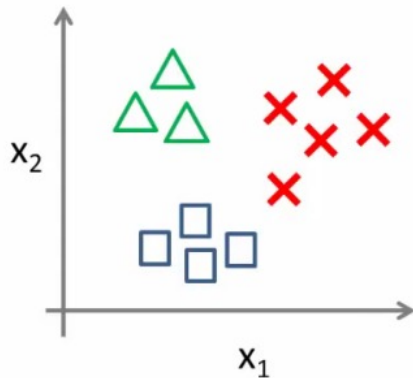
$$J^{(i)} = \begin{cases} -\log \hat{y}^{(i)} & \text{si } y^{(i)} = 1 \\ -\log(1 - \hat{y}^{(i)}) & \text{si } y^{(i)} = 0 \end{cases}$$

$$J^{(i)} = -y^{(i)} \log \hat{y}^{(i)} - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

Logistic Regression not only allows binary classification of observations (spam/non-spam, malignant/benign, black/white...) but also classification in the case where we have multiple classes.

If you need to classify an observation into one category out of three. For example, classifying a newspaper article into one of three categories : Sports, High-Tech, or Politics. In this case, we talk about Multi-class classification. The label $Y \in \{1, 2, 3\}$. We can then use a generalization of the logistic function : the Softmax function (referred to as Softmax Regression).

Multi-class classification:



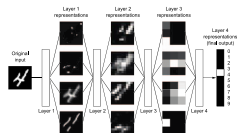
The Softmax function allows for returning at once the probability of belonging to each class. For example, if $Y \in \{1, 2, 3\}$, the Softmax function will return an array containing three probabilities of belonging to class 0, 1, or 2 respectively. The mathematical expression of Softmax for K classes is

$$\sigma(X) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$$

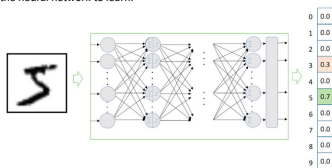
The result is a number giving the probability for an observation to belong to a class i , among the K different classes.

Deep Learning main steps I

Supervised Deep Learning

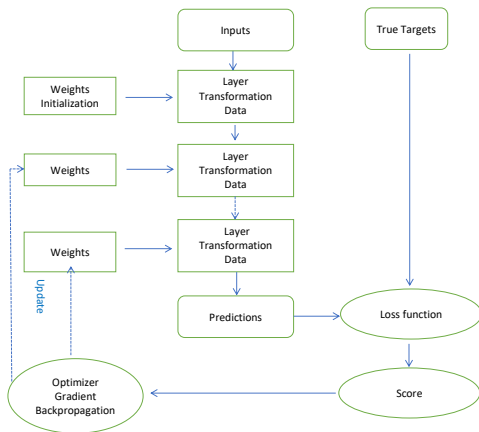


Initially, we have a training set: input values and corresponding output values (targets), To allow the neural network to learn:



If you have numbers from 0 to 9 then the neural network will learn the « features » of each number, in order to classify them correctly.

During the training step:
Until convergence the network will do:



What does the network ask for?

- 1) Set a distance (norm) measure for error estimation between the target and the calculated values before updating the weights. For this it is necessary to provide:
 - A Loss function.
 - A score that is the stopping criterion
- 2) Activation feature(s):

Without an activation function, the dense layer would consist of two linear operations: a scalar product and an addition.

⇒ The dimension of the space of the layers is too small and would not benefit from several layers of representations.

To access to a much richer space that would benefit from deep representations, you need a non-linearity via an activation function.

The "relu" function is the most popular activation function in deep learning, but there are many other candidates.

There are 2 types of general problem classes: Regression and classification.

Vocabulary:

- Layer: a layer is a module that takes as input one or more tensors (numbers, 2D tables, 3D, ...) and provides one or more tensors. The layers contain the data to be processed, and usually the weights of the network as well.
- Targets: values from the dataset known from the ground-truth and that we must approximate at the end of the training phase.
- Predictions: values obtained at the output by the neural network during its training.
- Loss function: measures the distance between the desired solution (Targets) and the solution obtained during training (Predictions). A multi-output neural network can admit a different output loss function.
- Optimizer: It determines how the neural network is updated. We use the gradient descent algorithm or one of its variants (SGD, Adams, etc.), the computations are done thanks to the backpropagation of the gradient. The optimizer computes the weights of the neural network.
- Weights: they are the ones who "tune" the neural network to obtain good predictions, i.e., good approximations of the targets. Once the weight vector is found, the network will predict a good ground-truth estimation.
- Mini-batch: a small set of samples that are processed simultaneously by the model. The number of samples is often a power of 2, to facilitate memory allocation. During training, a mini-batch is used to calculate a single update of the model weights per gradient descent.
- Epochs (epochs): number of iterations ("passes") on all the samples of the learning base during the gradient descent. Each "pass" is called epoch.
- Hyperparameter: parameter, external to the training process, it is fixed before the learning process and remains static during training (eg batch size, epoch, learning rate etc.).

How do I choose the activation function and the loss function?

Type of problem	Activation function	Loss function
Binary classification	Sigmoid	Binary Crossentropy Cross-entropy
Multi-class classification, single label	Softmax	Categorical Crossentropy Categorical cross-entropy
Multi-class, multi-label classification	Sigmoid	Binary Crossentropy
Regression to any value	Nothing	MSE Mean quadratic error
Regression to binary values	Sigmoid	MSE or Binary Crossentropy

